

Module 5 (Gestion de Caisse & Opérations de Guichet) - Implementation Learning

This is a practical reading guide for Module 5.

It explains how cash/teller behavior is implemented today and why controls are structured this way.

Primary backlog source:

- ``backlogs/module-5-cash-operations-completion-backlog.md``

Primary route surface:

- ``routes/api/v1/accounting.php`` (denominations, tills, teller sessions, teller transactions, reconciliations)

1) What this module owns

Module 5 owns daily cash operations:

1. Till configuration (who can operate which till, under what limits).
2. Teller session lifecycle (open/close).
3. Front-office cash flows (deposit, withdrawal, reversal).
4. Manual cash-originated journal operations.
5. End-of-day reconciliation and billeteage.
6. Cash control integration in batch close checks.

Key boundary:

- Module 5 handles operational cash workflows.
- Financial posting still uses Module 3 journal structures and controls.

2) Implemented entry points

From ``routes/api/v1/accounting.php``:

- Denominations:
 - ``GET/POST/PATCH /denominations``
- Tills:
 - ``GET/POST/PATCH /tills``
- Teller sessions:
 - ``GET/POST /teller-sessions``
 - ``POST /teller-sessions/{id}/close``
- Teller transactions:
 - ``POST /teller-sessions/{id}/deposits``
 - ``POST /teller-sessions/{id}/withdrawals``
 - ``POST /teller-transactions/{id}/reverse``
 - ``POST /teller-sessions/{id}/manual-journal-entries``
- Reconciliation:
 - ``GET/POST /teller-sessions/{id}/reconciliations``

3) Workflow architecture in code

Controllers are orchestration surfaces:

- `DenominationController`
- `TillController`
- `TellerSessionController`
- `TellerTransactionController`
- `TillReconciliationController`

Core business workflows are in:

- `app/Application/CashOperations/TellerSessionWorkflow.php`
- `app/Application/CashOperations/TellerCashTransactionWorkflow.php`
- `app/Application/CashOperations/TellerManualJournalWorkflow.php`
- `app/Application/CashOperations/TellerTransactionWorkflowControllerAdapter.php`

Batch integration:

- `app/Application/BatchRuns/ExecuteCashCloseVerificationBatch.php`

4) Epic-by-epic implementation summary

Epic 1 - Till Configuration Completion

What is implemented:

- Till records include operational fields (ledger link, daily state, limits, denomination requirement, nature, assignment).
- Validation enforces compatible active ledger account and same-agency constraints.
- Teller assignment rules prevent unsafe active multi-assignment patterns.

Why it matters:

- Cash control starts with correct till topology. Bad till setup creates downstream control failures.

Epic 2 - Teller Session Lifecycle

What is implemented:

- Open session requires active till + valid teller context + opening declaration.
- Close session requires closing declaration and blocks on unresolved states.
- Close-time checks prevent silent carry-forward when reconciliation/control conditions are not met.

Why it matters:

- Session boundaries are the legal/operational envelope for teller cash movements.

Epic 3 - Teller Transactions

What is implemented:

- Cash deposit and withdrawal endpoints with idempotency.
- Account availability and till-limit checks before posting.
- Reversal workflow linked to original teller transaction.
- Journal linkage for accounting traceability.

Why it matters:

- Prevents duplicate posting and ensures every cash movement is auditable and reversible through controlled paths.

Epic 4 - Manual Cash-Originated Journal Operations

What is implemented:

- Manual teller-originated journal path supports balanced double-entry operations.
- Unbalanced entries are rejected.
- Journal still follows posting/approval controls.

Why it matters:

- Gives operations flexibility for legitimate manual cases without bypassing accounting discipline.

Epic 5 - Reconciliation & Billetage

What is implemented:

- Reconciliation records include denomination lines and computed totals.
- Theoretical vs physical balance difference is computed and checked.
- Zero-tolerance difference policy is enforced in the approved default.
- Pending transactions block reconciliation close.

Why it matters:

- This is the control that converts “cash claims” into verifiable evidence.

Epic 6 - Cash Batch Integration

What is implemented:

- Cash-close verification integrates with batch framework.
- Day-close blocking conditions include open sessions, unresolved reconciliation, and pending cash states.

Why it matters:

- End-of-day cannot be marked complete while cash controls are failing.

5) Cross-module behavior (important for operations)

1. **With Module 3 (Accounting):**

Teller operations create or rely on journal structures; cash workflows do not bypass posting model.

2. **With Module 1 (Batch):**

Cash verification is part of close-control batch checks.

3. **With Module 4 (Credit):**

Loan cash disbursement channel uses open teller session/till controls from Module 5.

6) How to verify quickly

High-signal test suite:

- ``tests/Feature/Module5CashInfrastructureTest.php``

This suite covers:

- denomination and till contracts,
- teller session open/close,
- cash transactions + reversals,

- manual cash journal path,
- reconciliation and close blocking behavior.

Operationally: if these tests pass, the major Module 5 control surfaces are behaving as designed.